



4



16



2400 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

AVL Tree

عملي مشترك



RB Informatics; 20/5/2026

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

لقد تعرّفنا في المحاضرة السابقة على الـ Binary Search Tree كأحد هياكل البيانات المستخدمة لتنظيم المعلومات وتسهيل عمليات البحث والإضافة والحذف بكفاءة، لكن أداء الـ BST يتراجع عندما تصبح الشجرة غير متوازنة، مما يؤدي إلى انخفاض كبير في كفاءة العمليات، لذلك سنتناول في هذه المحاضرة AVL Tree. وهو نوع مطوّر من الـ BST يعمل على الحفاظ على توازن الشجرة وتحسين الأداء في مختلف العمليات.

لنفترض أن لدينا شجرة بحث ثنائية، ولكي نحدد إذا كانت متوازنة أم لا فإننا نعتمد على مفهوم التوازن الذي ينص على أن الفرق بين ارتفاع الفرع الأيمن وارتفاع الفرع الأيسر يجب أن يكون صغيراً

$$|Height(left) - Height(right)| \leq 1$$

أي الفرق يمكن ان يكون 1,0,-1

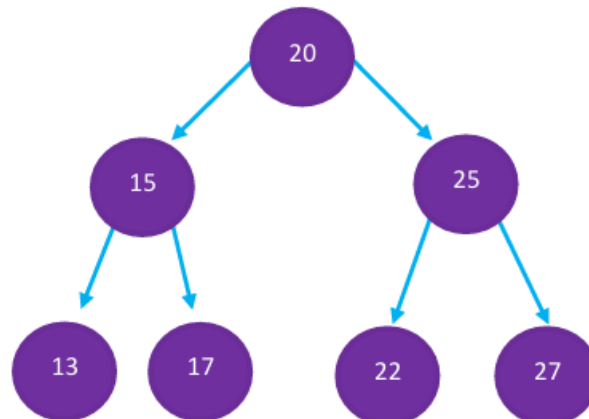
والمقصود بالارتفاع (Height) هو عدد الحواف في أطول مسار يبدأ من العقدة وينتهي عند إحدى الأوراق. لذلك، إذا عرفنا ارتفاع كل عقدة في الشجرة فسنتمكن من معرفة ما إذا كانت الشجرة بأكملها، أو أي Subtree داخلها، متوازنة أم لا. ومن هنا تبدأ فكرة موازنة شجرة البحث الثنائية، حيث سنقوم بتخزين قيمة إضافية في كل عقدة تمثل ارتفاعها، حتى نستطيع متابعة حالة التوازن بشكل مستمر.

Height

Height : 2

Height : 1

Height : 0



AVL Tree: هي نوع من أشجار البحث الثنائية التي تحافظ على توازنها بشكل تلقائي. ويعتمد هذا التوازن على حساب الفرق بين ارتفاع الابن الأيمن و ارتفاع الابن الأيسر لكل عقدة في الشجرة، بحيث يجب أن يبقى الفرق بينهما ضمن المجال $[-1, 1]$ ويُطلق على هذا الفرق اسم الـ Balance Factor.

فإذا كانت قيمة الـ Balance Factor لكل عقدة تقع ضمن هذا المجال، نعتبر أن الشجرة تحقق شرط التوازن الخاص بالـ AVL Tree، كما هو موضح في الشجرة المرسومة بالأعلى حيث تكون قيمة الـ Balance Factor لجميع العقد مساوية لـ 0.

بعد أن تعرّفنا على كيفية تحديد توازن الشجرة باستخدام الـ Height والـ Balance Factor، ننتقل الآن إلى الطريقة التي تستخدمها شجرة AVL Tree لاستعادة التوازن عند حدوث خلل:

فعندما يصبح أحد الفرعين أطول من الآخر، يتم تعديل ترتيب بعض العقد من خلال عملية تسمى **Rotation أو التدوير**، مع الحفاظ على خصائص شجرة البحث الثنائية.

ويوجد عدة أنواع من الـ Rotations مثل: Left Rotation و Right Rotation و Left-Right Rotation و Right-Left Rotation.

ملاحظة قبل البدء بالشرح :

المثلثات تشير لـ SUBTREES والدوائر تشير لعقد .

الدوران اليساري (Left Rotation)

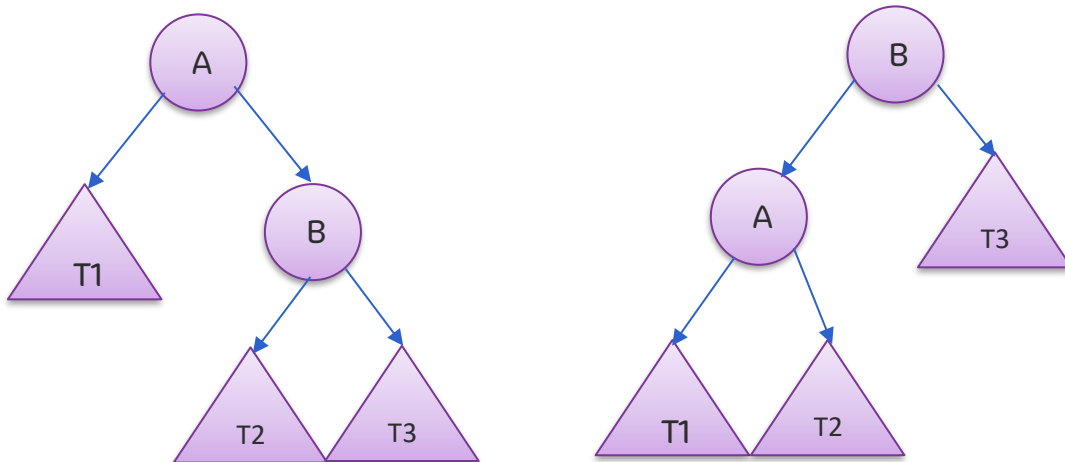
لنفترض أن لدينا عقدة اسمها A، ولها ابن أيمن اسمه B. إذا أردنا إجراء دوران أيسر على A، فإننا نعتبر B هو محور الدوران.

تتحرك العقدة A إلى اليسار لتصبح الابن الأيسر لـ B، بينما يبقى B في الأعلى كجذر للجزء الذي نعمل عليه.

لكن ماذا يحدث للفرع T2 (الذي كان الابن الأيسر لـ B)؟

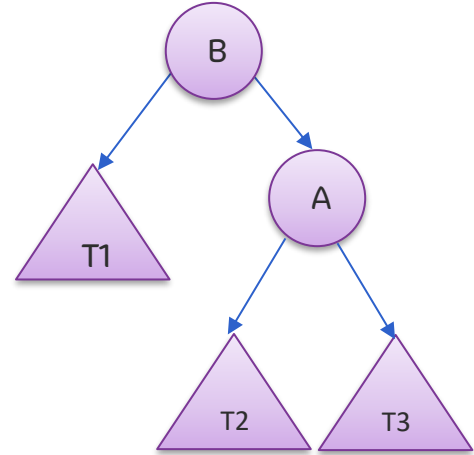
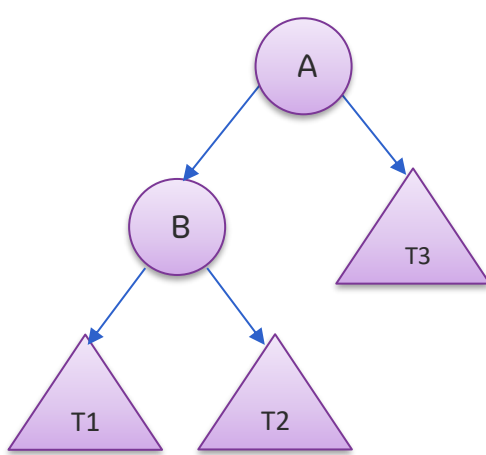
هذا الفرع ينتقل ليصبح الابن الأيمن لـ A، لأن جميع قيم T2 أكبر من A و أصغر من B، وهذا يتوافق تمامًا مع ترتيب شجرة البحث الثنائية.

بهذا الشكل نكون قد أعدنا توزيع العقد بحيث يصبح الفرع الأيمن أقل طولًا، ويعود التوازن.



الدوران اليميني (Right Rotation)

هذه العملية هي العكس تمامًا للدوران الأيسر. إذا كانت لدينا عقدة A ولها ابن أيسر B، فإن B يصبح محور الدوران. تتحرك A إلى اليمين لتصبح الابن الأيمن لـ B، بينما يبقى B في الأعلى. أما الفرع T2 (الذي كان الابن الأيمن لـ B)، فينتقل ليصبح الابن الأيسر لـ A، لأن جميع قيم T2 أصغر من A وأكبر من B، وهذا يحافظ على خصائص الـ BST.



ملاحظة: بعد كل عملية تدوير يجب تحديث قيمة الارتفاع للعقد التي تغير موقعها، لأن شجرة AVL تعتمد على الارتفاع لتحديد حالة التوازن.

العلاقة بين الدورانين:

الدوران الأيسر والأيمن هما عمليتان متعاكستان تمامًا، وكل واحدة منهما تستخدم لمعالجة حالة عدم توازن معينة. فهم هاتين العمليتين ضروري قبل الانتقال إلى نوعي الدورانات في الأسفل لأن هذه الدورانات تعتمد على دمج الدورانين الأساسيين.

Left-Right Rotation (LR)

عندما لا يكفي الدوران الأيسر وحده ولا الدوران الأيمن وحده لإعادة توازن الشجرة، نستخدم ما يسمى الدوران المزدوج Left-Right. هذا النوع يحدث عندما يكون الخلل في الجهة اليسرى للعقدة A، ولكن الابن الأيسر يكون مائلًا إلى اليمين لذلك نحتاج خطوتين متتاليتين.

• الخطوة الأولى: Left Rotation على الابن الأيسر (B)

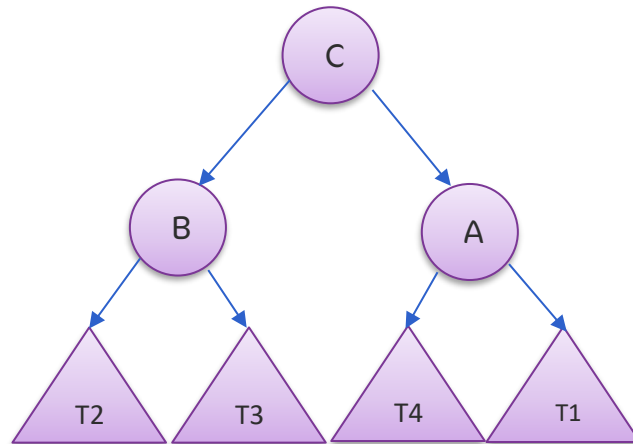
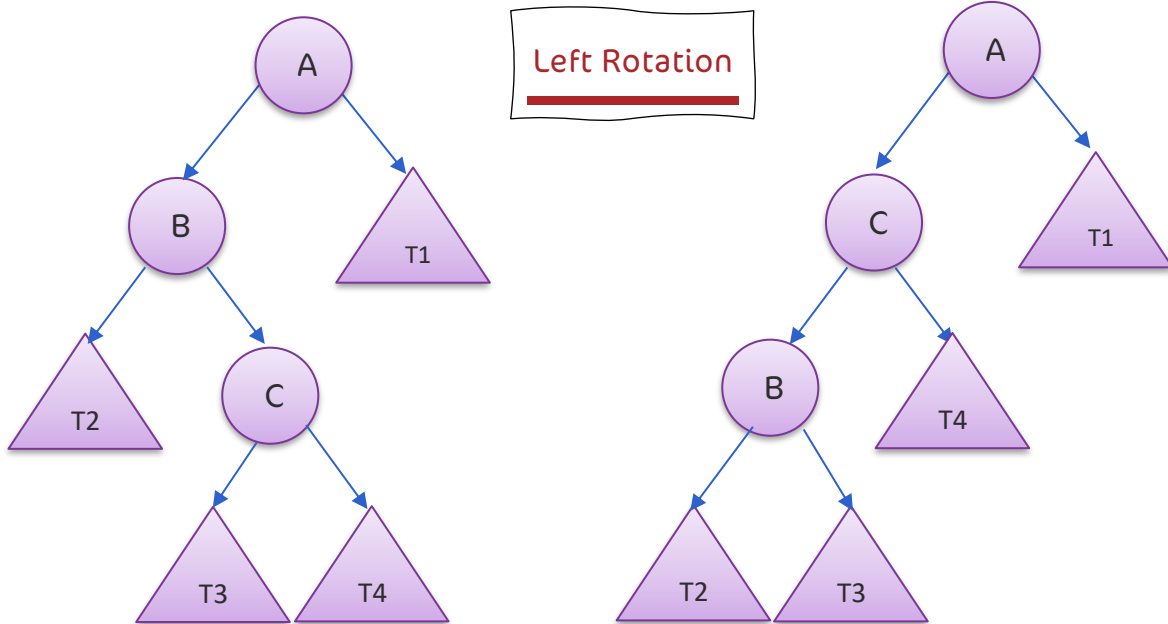
ننظر أولاً إلى ابن A الأيسر وهو B، نلاحظ أن B لديه ابن أيمن اسمه C، وهذا هو سبب عدم التوازن.

لذلك نقوم بعملية Left Rotation على B:

▪ يصبح C هو المركز الجديد بدلاً من B.

▪ نحرك B ليصبح ابناً أيسراً لـ C.

- الفرع T2 (الذي كان الابن الأيسر لـ C) ينتقل ليصبح الابن الأيمن لـ B، لأن جميع قيم T2 أكبر من B وأصغر من C.
- بهذه الخطوة نكون قد أصلحنا الميل داخل الفرع الأيسر.
- الخطوة الثانية: Right Rotation على العقدة A نفسها بعد أن أصبح C هو الابن الأيسر الجديد لـ A، ننتقل للخطوة الثانية.
- نقوم الآن بـ Right Rotation على A:
- يصبح C هو الجذر الجديد لهذا الجزء من الشجرة.
- تتحرك A ليصبح ابناً أيمن لـ C.
- الفرع T3 (الذي كان الابن الأيمن لـ C) ينتقل ليصبح الابن الأيسر لـ A، لأن جميع قيم T3 أكبر من C وأصغر من A.



هذا النوع يستخدم عندما يكون الخلل في الجهة اليمنى للعقدة A لكن الابن الأيمن يكون مائلا لليسار لذلك لا يكفي دوران واحد فقط ونحتاج الى دوران مزدوج لإعادة التوازن ويتكوّن من خطوتين متتاليتين:

• الخطوة الأولى: Right Rotation على الابن الأيمن (B)

نبدأ بالنظر إلى الابن الأيمن للعقدة A وهو B، نلاحظ أن B لديه ابن أيسر C، وهذا هو سبب عدم التوازن. لذلك نقوم بعملية Right Rotation على B:

- تصبح C هي المركز الجديد بدلاً من B.
- تتحرك B لتصبح ابناً أيمن لـ C.
- الفرع T4 (الذي كان الابن الأيمن لـ C) ينتقل ليصبح الابن الأيسر لـ B، لأن جميع قيم T4 اكبر من C وأصغر من B

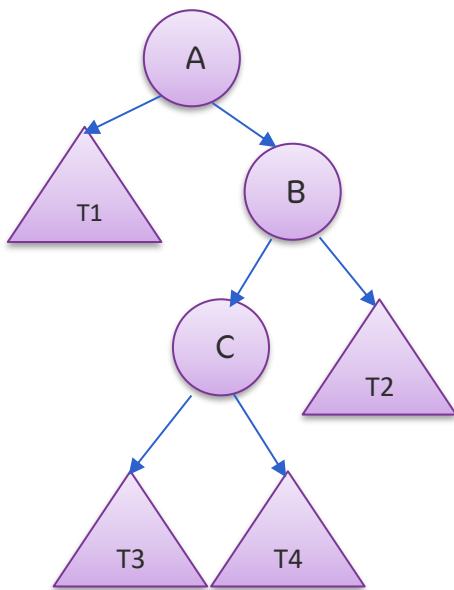
بهذا نكون قد أصلحنا الميل داخل الفرع الأيمن.

• الخطوة الثانية: Left Rotation على العقدة A نفسها

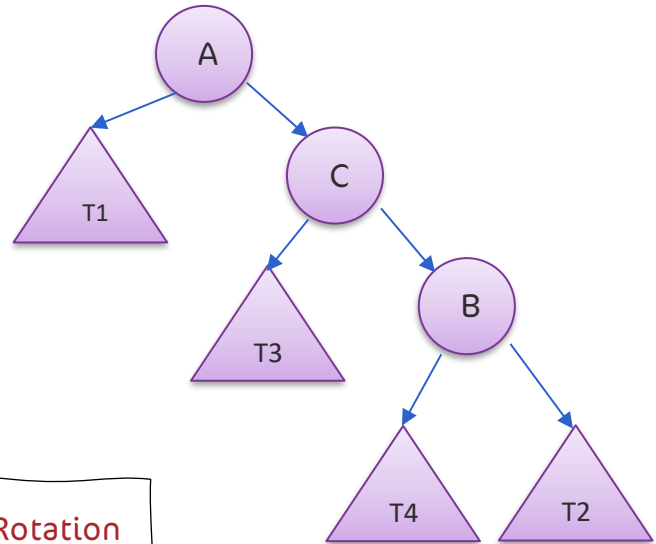
بعد أن أصبحت C هي الابن الأيمن الجديد لـ A، ننتقل للخطوة الثانية.

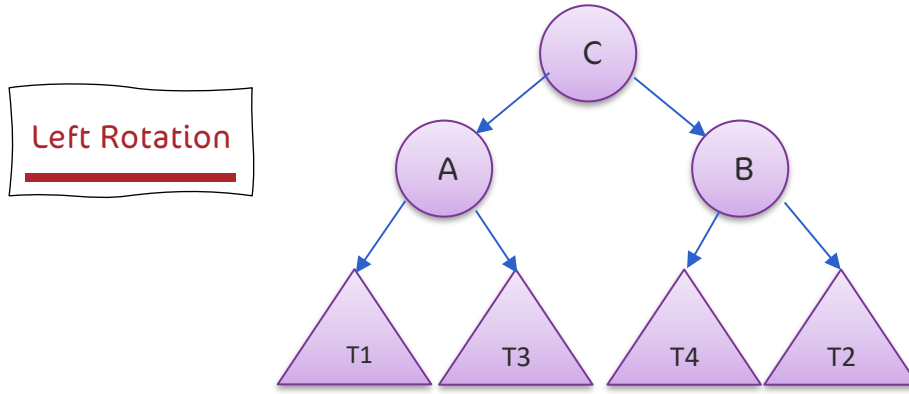
نقوم الآن بـ Left Rotation على A:

- تصبح C هي الجذر الجديد لهذا الجزء من الشجرة.
- تتحرك A لتصبح ابناً أيسر لـ C.
- الفرع T3 (الذي كان الابن الأيسر لـ C) ينتقل ليصبح الابن الأيمن لـ A، لأن جميع قيم T3 اكبر من A وأصغر من C.



Right Rotation





متى نستخدم كل نوع من الدورانات في شجرة AVL؟
 عندما تصبح شجرة AVL غير متوازنة، فهذا يعني أن الفرق بين ارتفاع الفرع الأيسر والفرع الأيمن عند عقدة معينة أصبح أكبر من 1 و أصغر من -1 .
 ولكي نعرف نوع الدوران المناسب، ننظر أولاً إلى الجهة التي حدث فيها الخلل، ثم نفحص ابن تلك الجهة لمعرفة ما اذا كان عد التوازن في اليمين او اليسار .
 نفترض أن لدينا عقدة اسمها C، ولها ابن أيسر A وابن أيمن B.

ثانياً: إذا كان الميل في الجهة اليمنى من C
Case (RR) ميل خارجي يمين-يمين

يحدث عندما يكون:

- الفرع الأيمن لـ C أعلى من الأيسر
- وابن اليمين B يميل أيضاً لليمين أي :
 $\text{Height}(B.\text{Right}) > \text{Height}(B.\text{Left})$
- الحل هو Left Rotation على C.

Case (RL) ميل داخلي يمين-يسار

يحدث عندما يكون:

- الميل في الجهة اليمنى من C
- لكن ابن اليمين B يميل لليسار أي :
 $\text{Height}(B.\text{Left}) > \text{Height}(B.\text{Right})$
- الحل يكون دورانا مزدوجاً:
- 1. Right Rotation على B
- 2. Left Rotation على C

أولاً: إذا كان الميل في الجهة اليسرى من C
Case (LL) ميل خارجي يسار-يسار

يحدث عندما يكون:

- الفرع الأيسر لـ C أعلى من الأيمن
- وابن اليسار A يميل أيضاً لليسار، أي :
 $\text{Height}(A.\text{Left}) \geq \text{Height}(A.\text{Right})$
- في هذه الحالة نستخدم Right Rotation على C.

Case (LR) ميل داخلي يسار-يمين

يحدث عندما يكون:

- الميل في الجهة اليسرى من C
- لكن ابن اليسار A يميل لليمين أي :
 $\text{Height}(A.\text{Right}) > \text{Height}(A.\text{Left})$
- هنا نحتاج دورانا مزدوجاً:
- 1. Left Rotation على A
- 2. Right Rotation على C

هذه الحالات الأربع التي سنقوم باستخدامها في بناء شجرة ال AVL

■ إذا كان الميل في اليسار:

ابن اليسار يميل للييسار $LL \rightarrow \text{Right Rotation}$

ابن اليسار يميل لليمين $LR \rightarrow \text{Left} \rightarrow \text{Right}$

■ إذا كان الميل في اليمين:

ابن اليمين يميل لليمين $RR \rightarrow \text{Left Rotation}$

ابن اليمين يميل للييسار $RL \rightarrow \text{Right} \rightarrow \text{Left}$

الحالة	المكان غير المتزن	اتجاه المشكلة	نوع الدوران المستخدم
LL	الييسار	الييسار	Right Rotation
LR	الييسار	الييمين	Left then Right
RR	الييمين	الييمين	Right Rotation
RL	الييمين	الييسار	Right then Left

توازن شجرة AVL وتحديد نوع الدوران

بعد أي عملية إضافة أو حذف داخل شجرة البحث الثنائية، قد تفقد الشجرة توازنها، لذلك نستخدم عمليات تسمى Rotations لإعادة التوازن من جديد.

فكرة الـ Rotation هي إعادة تنظيم العقد بطريقة تحافظ على خصائص شجرة البحث الثنائية وفي نفس الوقت تجعل ارتفاع الجهتين متقارباً.

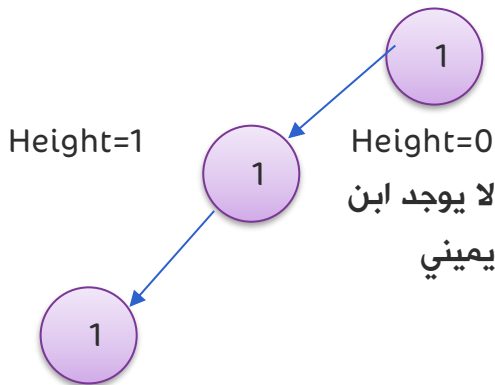
كيف نعرف أن هناك خللاً؟

❖ حتى نحدد إذا كانت المشكلة في الشجرة اليمنى أو اليسرى، نحسب الفرق بين ارتفاع الجهة اليسرى وارتفاع الجهة اليمنى للعقدة:

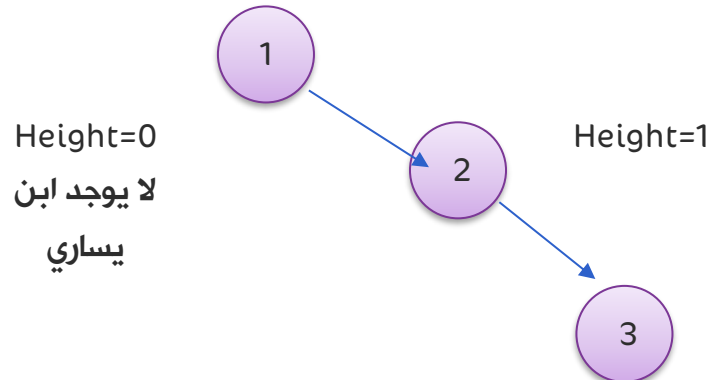
Balance factor: Height(Left)-Height(Right)

❖ تفسير النتائج:

- إذا كانت النتيجة: 1, 0, -1 فهذا يعني أن العقدة ما زالت متوازنة ولا يوجد أي مشكلة.
- إذا كانت النتيجة أكبر من 1، فهذا يعني أن ارتفاع الجهة اليسرى أكبر من ارتفاع الجهة اليمنى بأكثر من 1، وبالتالي يكون الخلل في اليسار.
- إذا كانت النتيجة أصغر من -1، فهذا يعني أن ارتفاع الجهة اليمنى أكبر من ارتفاع الجهة اليسرى، وبالتالي يكون الخلل في اليمين.



$$\text{Height(left)} - \text{Height(right)} = 2 - 0 = 2$$



$$\text{Height(left)} - \text{Height(right)} = 0 - 2 = -2$$

كيف نحدد نوع الدوران؟

بعد أن نحدد جهة الخلل، نحتاج لمعرفة نوع الحالة حتى نختار الـ Rotation المناسب.
نفترض أن العقدة غير المتوازنة اسمها X.
الخطوة الأولى كما ذكرنا، حساب:

$$\text{Height(Left)} - \text{Height(Right)}$$

لكن هذا لا يكفي، لأننا نحتاج لمعرفة هل الميل داخلي أم خارجي.
إذا كان الخلل في اليسار

ننظر إلى الابن الأيسر للعقدة، ونحسب:

$$\text{Height(ROOT.LEFT.LEFT)} - \text{Height(ROOT.LEFT.RIGHT)}$$

الحالة الثانية: Left - Right (LR)

إذا كانت النتيجة أصغر من 0

$$\text{Height(ROOT.Left.Right)} >$$

$$\text{Height(ROOT.Left.Left)}$$

فهذا يعني أن الجهة اليمنى للابن الأيسر أعلى من
الجهة اليسرى أي أن الميل داخلي باتجاه اليمين.

الحل:

نستخدم:

Left-Right Rotation

أي:

Right Rotation ثم Left Rotation

الحالة الأولى: Left - Left (LL)

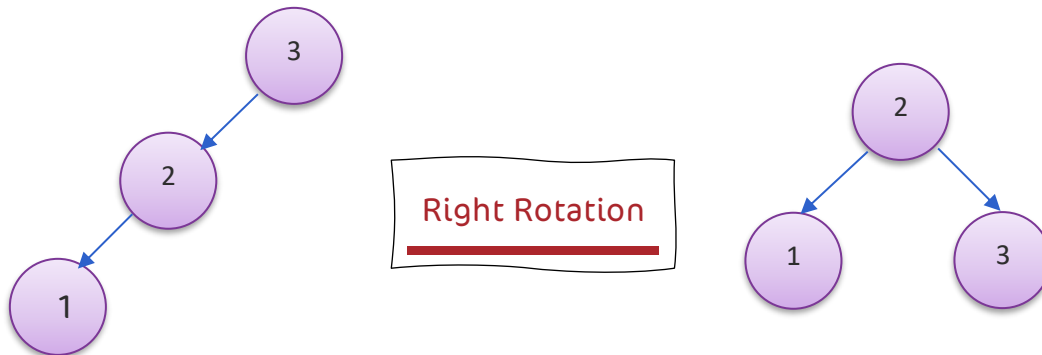
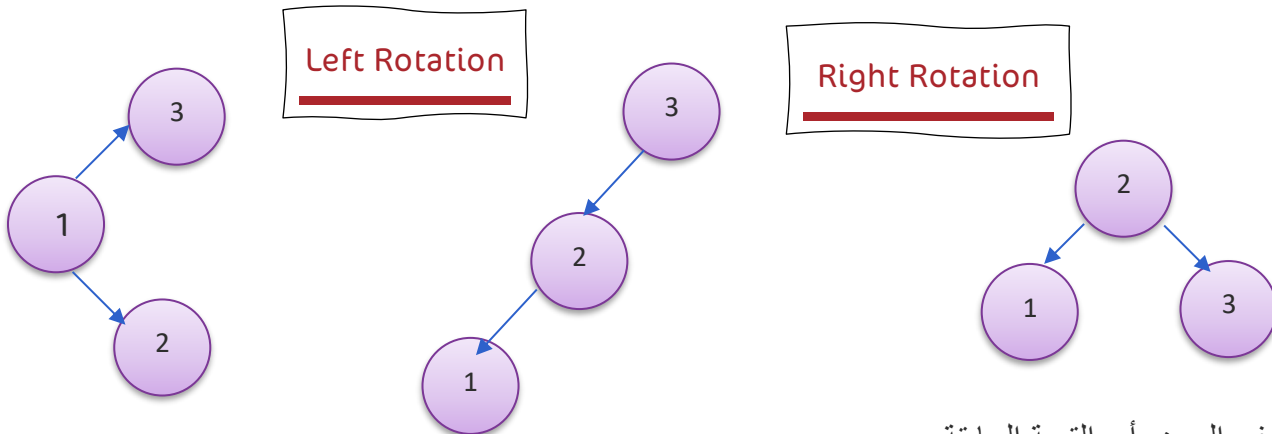
إذا كانت النتيجة أكبر أو تساوي 0

فهذا يعني أن الابن الأيسر مائل أيضًا إلى اليسار، أي
أن ارتفاع الجهة اليسرى لديه أكبر أو يساوي ارتفاع
الجهة اليمنى.

الحل:

نستخدم: Right Rotation على العقدة غير
المتوازنة.

بعد تنفيذ الدوران يتم تحديث قيم الـ Height للعقد
المتأثرة.

الحالة الأولى: LL(left left) الدوران يميني:الحالة الثانية: LR(left right) دوران يساري ثم يميني:

إذا كان الخلل في اليمين أي القيمة السابقة
ننظر إلى الابن الأيمن للعقدة، ونحسب:

$$\text{Height}(\text{ROOT}.\text{RIGHT}.\text{LEFT}) - \text{Height}(\text{ROOT}.\text{RIGHT}.\text{RIGHT})$$

الحالة الرابعة: Right Left (RL)

إذا كانت النتيجة أكبر من 0

$$\text{Height}(\text{ROOT}.\text{Right}.\text{Left}) >$$

$$\text{Height}(\text{ROOT}.\text{Right}.\text{Right})$$

فهذا يعني أن الجهة اليسرى للابن الأيمن أعلى من
الجهة اليمينية، أي أن الميل داخلي باتجاه اليسار

الحل :

نستخدم:

Right-Left Rotation

أي:

1. Right Rotation على الابن اليمين

2. ثم Left Rotation على العقدة غير المتوازنة

الحالة الرابعة: Right Right(RR)

إذا كانت النتيجة أصغر أو تساوي 0

$$\text{Height}(\text{ROOT}.\text{Right}.\text{Right}) \geq$$

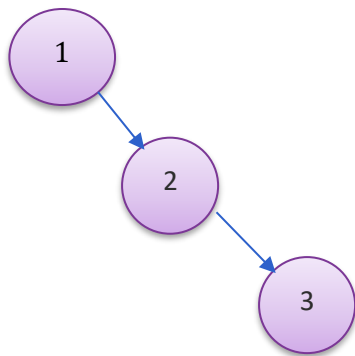
$$\text{Height}(\text{ROOT}.\text{Right}.\text{Left})$$

فهذا يعني أن الجهة اليمينية للابن الأيمن أكبر أو
تساوي الجهة اليسرى ، أي أن الميل خارجي باتجاه
اليمين

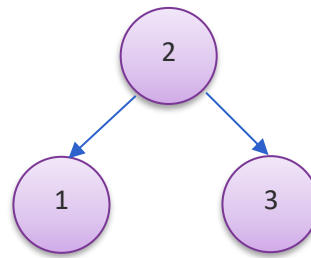
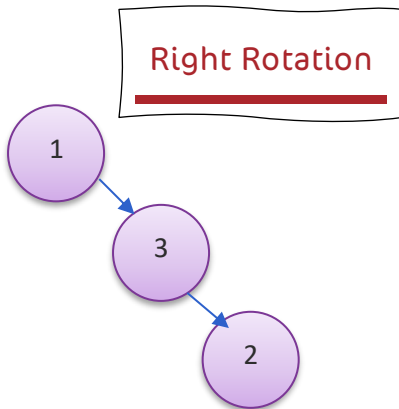
الحل:

نستخدم:

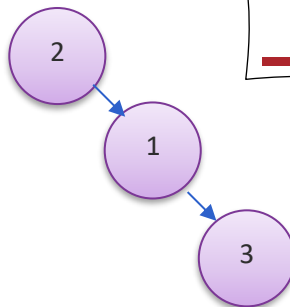
Left Rotation على العقدة غير المتوازنة.

الحالة الثالثة: RR(right right) دوران يساري:

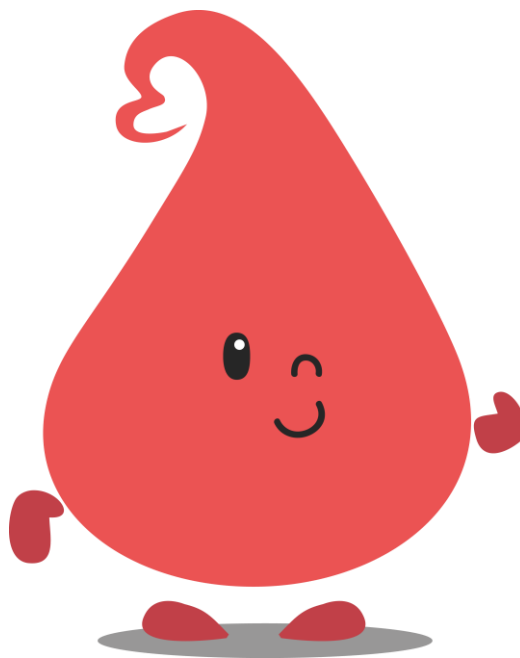
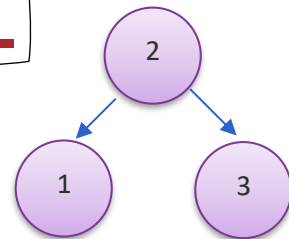
left Rotation

الحالة الرابعة: RL(right left) دوران يميني ثم يساري:

Right Rotation



left Rotation



:Insert

عند إضافة عنصر جديد في شجرة AVL ، تتم عملية الإدراج أولاً كما في شجرة البحث الثنائية BST ، حيث ننتقل يساراً إذا كانت القيمة أصغر من قيمة العقدة الحالية ويميناً إذا كانت أكبر، وعند الوصول إلى موقع فارغ يتم إنشاء عقدة جديدة. بعد إتمام الإضافة، تبدأ مرحلة التحقق من التوازن.

شجرة AVL تعتمد على قيمة الارتفاع Height لكل عقدة، لذلك بعد كل عملية إدراج نقوم بما يلي:
تحديث الارتفاع للعقدة الحالية اعتماداً على ارتفاع أبنائها.

حساب قيمة الـ Balance Factor باستخدام العلاقة:

$$[\text{Balance} = \text{Height}(\text{left}) - \text{Height}(\text{right})]$$

هذه القيمة تخبرنا ما إذا كانت العقدة متوازنة أم لا:

إذا كانت القيمة 1 أو 0 أو -1 → العقدة متوازنة ولا حاجة لأي تعديل.

إذا أصبحت القيمة أكبر من 1 فهذا يعني أن الخلل في الجهة اليسرى.

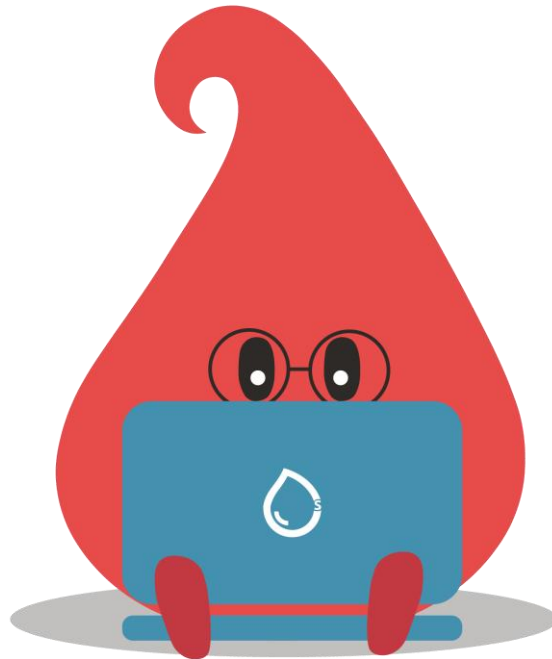
وإذا أصبحت أصغر من -1 فهذا يعني أن الخلل في الجهة اليمنى.

عند اكتشاف الخلل، لا يكفي معرفة الجهة فقط، بل يجب معرفة اتجاه الميل داخل تلك الجهة، وذلك من خلال مقارنة ارتفاعات أبناء العقدة.

وبناءً على اتجاه الميل، نحدد أي نوع من الدورانات الأربعة يجب تطبيقه لإعادة التوازن (LL - LR - RR - RL) ،

وهي الحالات التي تم شرحها سابقاً في المحاضرة.

بعد تنفيذ الدوران المناسب، تعود الشجرة إلى وضعها المتوازن، وتستمر عملية الإضافة بكفاءة.



:Delete

تتم عملية الحذف في شجرة AVL بنفس الخطوات الأساسية المستخدمة في شجرة البحث الثنائية. BST
فعند حذف عنصر معين، نبحث عنه أولاً داخل الشجرة، ثم نتعامل مع إحدى الحالات الثلاث المعروفة:

العقدة ورقة (لا أبناء لها)

يتم حذفها مباشرة.

العقدة لها ابن واحد فقط

نستبدلها بابنها مباشرة لأن هذا يحافظ على ترتيب شجرة البحث.

العقدة لها ابنان

في هذه الحالة نبحث عن أصغر قيمة في الفرع الأيمن (Successor)،

ننسخ هذه القيمة إلى العقدة المراد حذفها،

ثم نحذف تلك القيمة من الفرع الأيمن.

بعد تنفيذ الحذف، تبدأ الخطوة الأهم في شجرة AVL وهي إعادة التوازن.

فالحذف قد يغير ارتفاع أحد الفروع، مما يؤدي إلى اختلال التوازن عند بعض العقد.

لذلك، بعد كل عملية حذف نقوم بما يلي:

تحديث الارتفاع Height للعقدة الحالية.

حساب الـ Balance Factor لمعرفة ما إذا كانت العقدة ما زالت متوازنة.

إذا كانت قيمة الـ Balance خارج المجال {0، 1، -1} فهذا يعني أن الشجرة أصبحت غير متوازنة.

في هذه الحالة نحدد جهة الخلل (يسار أو يمين)، ثم ننظر إلى اتجاه الميل داخل تلك الجهة لمعرفة نوع الحالة

المناسبة من الحالات الأربع (LL - LR - RR - RL)، والتي تم شرحها سابقاً

بعد تطبيق الدوران المناسب، تستعيد الشجرة توازنها من جديد، وتعود جميع العمليات للعمل بكفاءة.

ملاحظة مهمة جداً:

أثناء عملية الإضافة يمكننا الاستفادة من القيمة الجديدة لمعرفة جهة الخلل.

إذا كانت القيمة المضافة أصغر من قيمة العقدة، فهذا يعني أنها أضيفت في اليسار، وبالتالي غالباً يكون الخلل في الجهة اليسرى.

أما إذا كانت القيمة المضافة أكبر من قيمة العقدة، فهذا يعني أنها أضيفت في اليمين، وبالتالي غالباً يكون الخلل في الجهة اليمنى.

وعلى هذا المبدأ سنستخدم أثناء كتابة الخوارزمية، حيث سيتم بعد كل عملية إضافة أو حذف التحقق من التوازن، ثم

استدعاء عملية الـ Rotation المناسبة لإعادة الشجرة إلى حالة التوازن

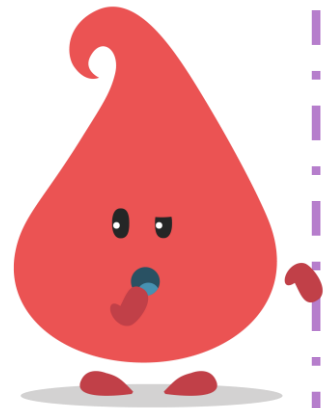
```
1 class AVL {
2   class NODE {
3     char key;
4     int height;
5     NODE left, right;
6     NODE(char key) {
7       left=right=null;
8       this.key =key;
9       height =0;
10    }
11  }
12
13  int height(NODE node) {
14    // returns the height of the actual node and -1 for nulls
15    return (node == null) ? -1 : node.height;
16  }
17
18  int getBalance(NODE node) {
19    // this will return (the height of the left tree - the height of the right tree)
20    // if the node is null return 0 since both children are null (height -1)
21    return (node ==null) ?0 : height(node.left) - height(node.right);
22  }
23
24  NODE rightRotate(NODE y) {
25    //left child of the original node
26    NODE x = y.left;
27    // the right child of the left child of the original node
28    NODE T2 = x.right;
29    // the right child of the left child of the original node is the original node
30    x.right = y;
31    // the new left child of the original node (T2)
32    y.left = T2;
33    // new heights
34    y.height = Math.max(height(y.left), height(y.right)) + 1;
35    x.height = Math.max(height(x.left), height(x.right)) + 1;
36    //this returns the left child (it will become the new root)
37    return x;
38  }
```

2

```

1 NODE leftRotate(NODE y) {
2     //right child of the original node
3     NODE x = y.right;
4     // the left child of the right child of the original node
5     NODE T2 = x.left;
6     // the left child of the right child of the original node is the original node
7     x.left = y;
8     // the new right child of the original node (T2)
9     y.right = T2;
10    // new heights
11    x.height = Math.max(height(x.left), height(x.right)) + 1;
12    y.height = Math.max(height(y.left), height(y.right)) + 1;
13    //this returns the right child (it will become the new root)
14    return x;
15 }

```



3

```

2 NODE insert(NODE node, char key) {
3     //common insertion
4     if (node == null) return new NODE(key);
5     if (key < node.key)
6         node.left = insert(node.left, key);
7     else if (key > node.key)
8         node.right = insert(node.right, key);
9     else
10        return node;
11    //calculate the new height
12    node.height = 1 + Math.max (height
(node.left),
13    height(node.right)) ;
14    //the difference between the heights of the
15    children
16    int balance=getBalance(node);
17    //right rotation case
18    if (balance>1&& key < node.left.key)
19        return rightRotate(node);

```

4

```

1 //left rotation case
2 if (balance<-1&&key>node.right.key)
3     return leftRotate(node);
4 //left-right rotation case
5 if (balance>1&&key>node.left.key) {
6     node.left = leftRotate(node.left);
7     return rightRotate(node);
8 }
9 //right-left rotation case
10 if (balance < -1&& key < node.right.key)
11 {
12     node.right =rightRotate(node.right);
13     return leftRotate(node);
14 }
15 return node;
16 }

```

```

1  NODE minValueNode(NODE node) {
2      NODE current = node;
3      while (current.left != null)
4          current = current.left;
5      return current;
6  }
7
8  NODE delete(NODE root, char key) {
9      // common deletion
10     if (root == null)
11         return root;
12     if (key < root.key)
13         root.left = delete(root.left, key);
14     else if (key > root.key)
15         root.right = delete(root.right, key);
16     else {
17         if((root.left == null) && (root.right == null))
18             return null;
19         else if ((root.left == null))
20             return root.right;
21         else if ((root.right == null))
22             return root.left;
23         else {
24             NODE temp = minValueNode(root.right);
25             root.key = temp.key;
26             root.right = delete(root.right, temp.key);
27         }
28     }
29 }

```

```

1 void inorder(NODE root) {
2     if (root != null) {
3         inorder(root.left);
4         System.out.print(root.key + " ");
5         inorder(root.right);
6     }
7 }
8 }

```

```

5 1 if (root == null)
2     return root;
3 //calculate the new height
4 root.height = 1 + Math.max
    (height(root.left), height(root.right));
6 //the difference between the heights of
    the children
7 int balance = getBalance(root);
8 //right rotation case
9 if (balance > 1 && getBalance(root.left) >= 0) {
10     return rightRotate(root);
11 }
12 //left-right rotation case
13 if (balance > 1 && getBalance(root.left) < 0) {
14     root.left = leftRotate(root.left);
15     return rightRotate(root);
16 }
17 //left rotation case
18 if (balance < -1 && getBalance(root.right) <= 0) {
19     return leftRotate(root);
20 }
21 //right-left rotation case
22 if (balance < -1 && getBalance(root.right) >
    0) {
23     root.right = rightRotate(root.right);
24     return leftRotate(root);
25 }
26 return root;
27 }

```



```

1 public class Main{
2 public static void main(String[] args) {      AVL tree = new AVL();
3 AVL.NODE root = null;      char[] values = {'H','I','J','B','A','E','C','F','D','G','K','L'};
4 for (char val : values) {
5 root = tree.insert(root, val);      tree.inorder(root);
6 System.out.println();      } }
7 }

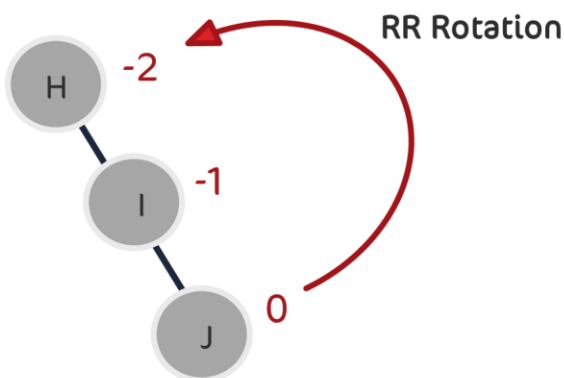
```

مثال عملي

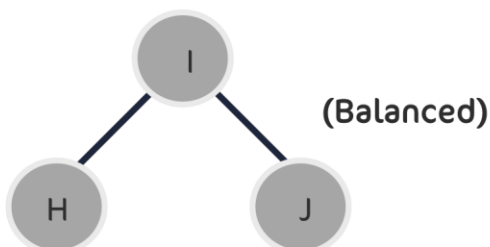
1. Construct an AVL tree having the following elements

H, I, J, B, A, E, C, F, D, G, K, L

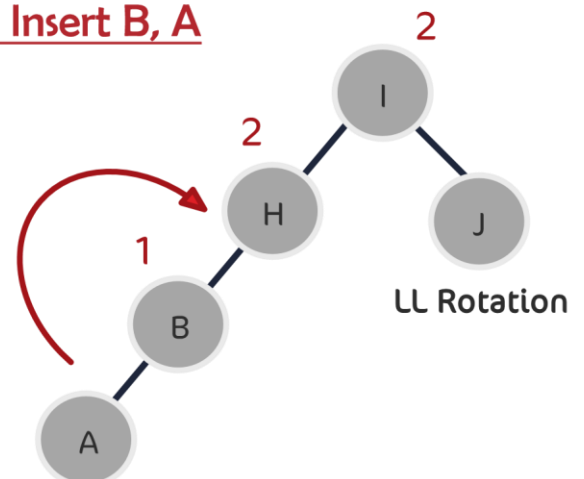
1. Insert H, I, J



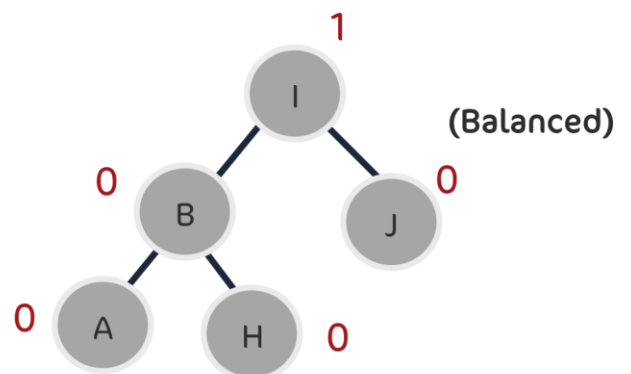
The resultant balance tree is:



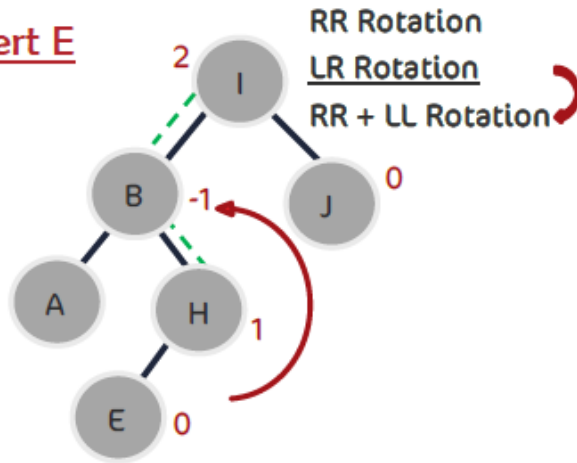
2. Insert B, A



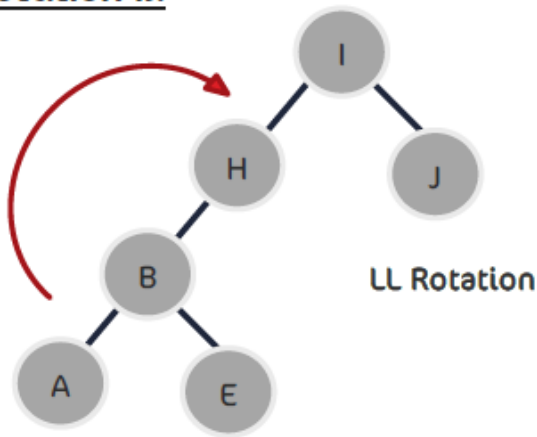
The resultant balance tree is:



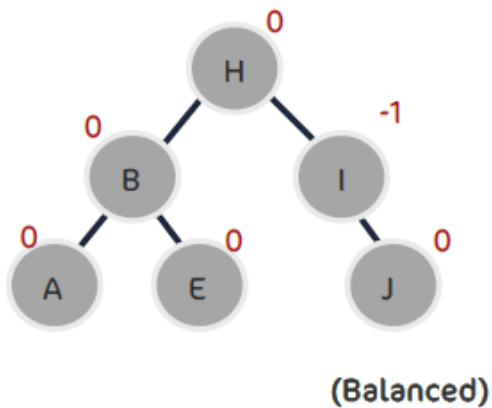
3. Insert E



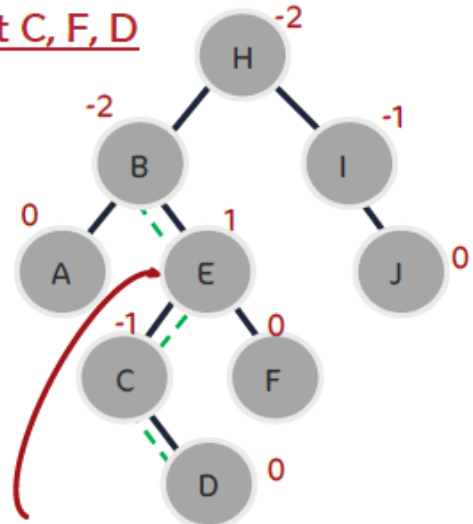
a) We first perform RR rotation on node B
The resultant balance tree after RR rotation is:



b) We first perform LL rotation on the node I
The resultant balance tree after LL rotation is:

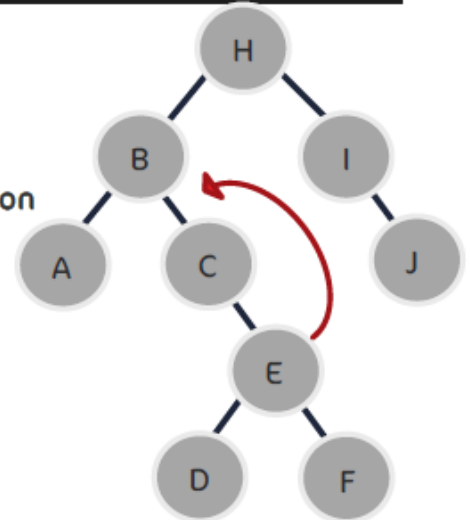


4. Insert C, F, D

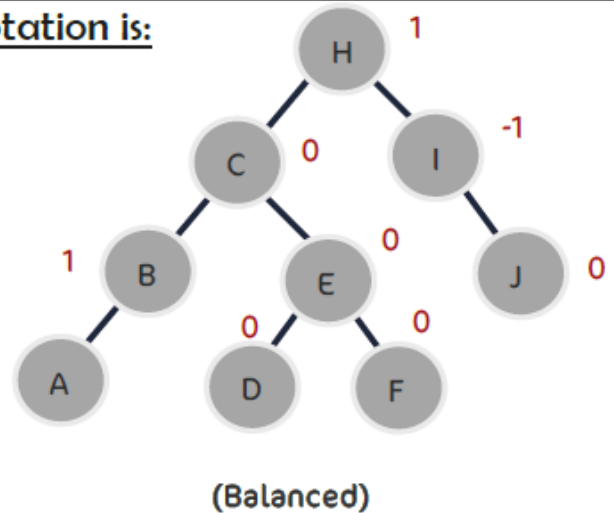


a) We first perform LL rotation on node E
The resultant balance tree after LL rotation is:

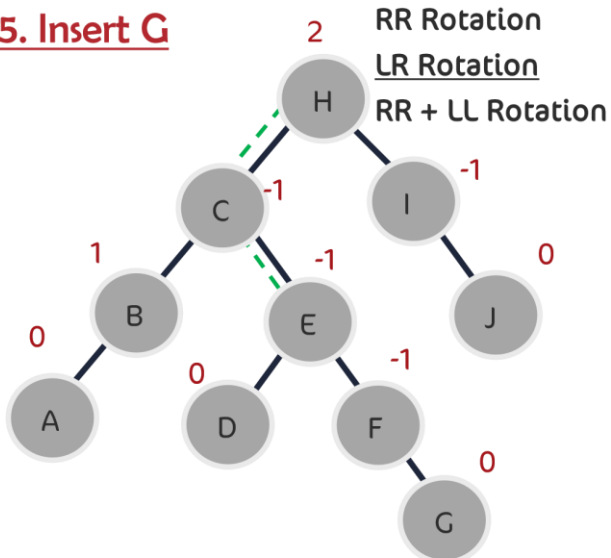
LL Rotation
 RL Rotation
 RR + LL Rotation



b) We then perform RR rotation on node B
The resultant balanced tree after RR rotation is:

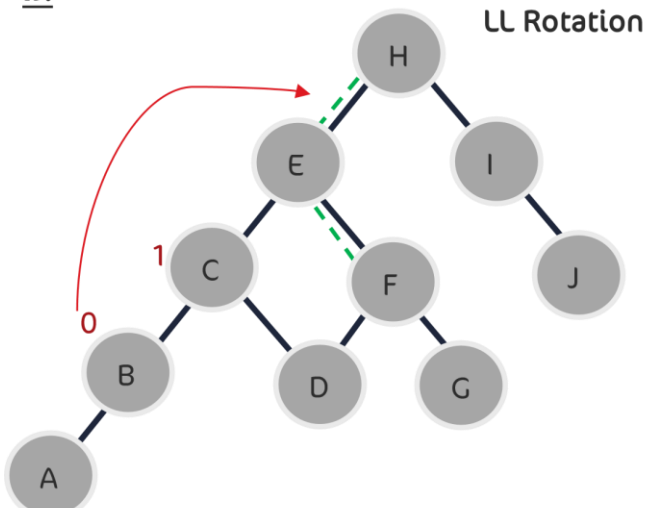


5. Insert G



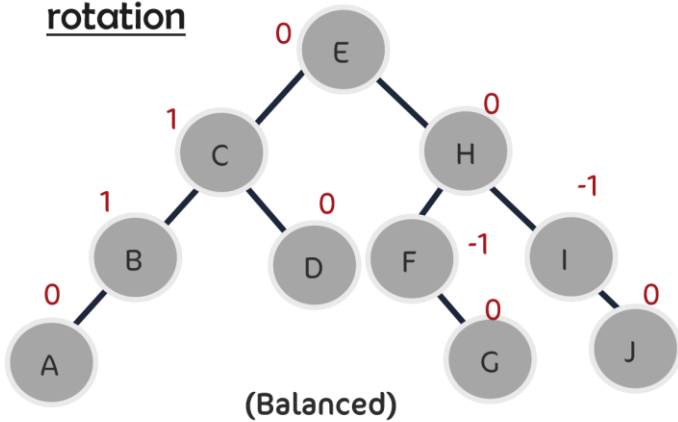
a) We first perform RR rotation on node C

The resultant tree after RR rotation is:

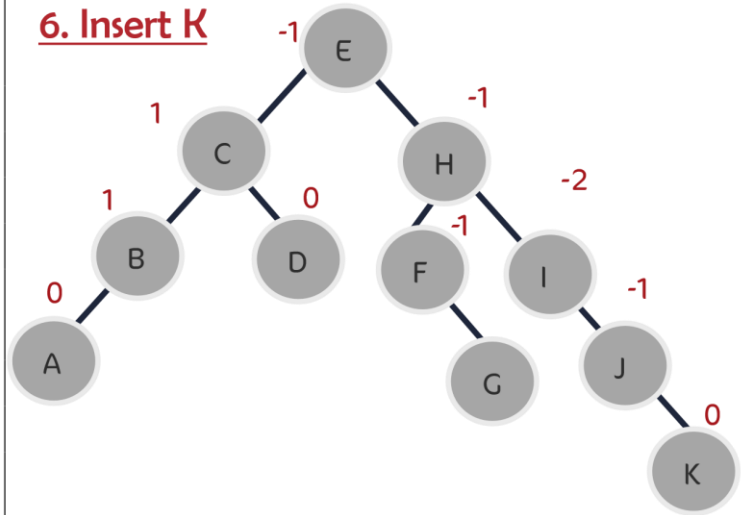


b) We then perform LL rotation on node H

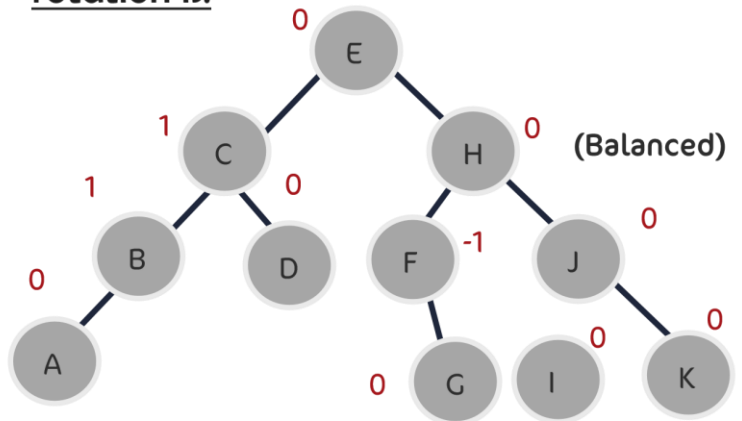
The resultant balanced tree after LL rotation



6. Insert K



The resultant balanced tree after RR rotation is:



7. Insert L

